

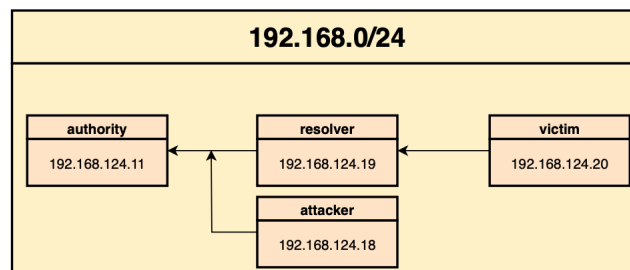
Network Security

Assignment: Application Layer

DNS Hijacking (50 points)

This assignment concerns an application server that regularly checks for updates. However, the update server is not properly secured, so if we can manage to redirect the update to our own malicious update server, we can exploit this process. In this exercise, we will target the DNS infrastructure to accomplish this.

The vulnerable DNS resolver will be running on 192.168.124.19. It periodically resolves the domain name `update-server.updateserver.corp` from its authoritative name-server at 192.168.124.11. The attacker has a man-in-the-middle position between the DNS resolver and the authoritative resolver and can observe all traffic passing through that link.



The responses from the authoritative name-server are served with a TTL of 15 seconds, and are therefore also cached for 15 seconds on the local name-server. In other words, every 15 seconds, the local server will forward the request to the authoritative DNS server.

Set up the environment using the commands below.

```
docker compose build
docker compose up -d
```

The *attacker* container has a mount configured, such that you can edit files directly on your computer in an editor of your choice and then access them within the container: files in the `solution` directory are mirrored in the container under `/solution`. To connect to the attacker container, execute the command below.

```
docker exec -ti assignment_5_dns_attacker bash
```

After finishing the assignment, you can destroy the environment using the command below.

```
docker compose down
```

Goal

You should produce a Python3 script that poisons the DNS cache of the local DNS server, by intercepting the DNS query and spoofing a response. After successfully poisoning the DNS cache, the script should exit. The cache should remain poisoned for a long period after the attack ends.

The script should accept 3 arguments: the DNS server we are targeting, the domain name of the DNS request, and the IP address we want the answer to be.

For instance:

```
python3 dns_hijack.py 192.168.124.19 update-server.updateserver.corp 1.2.3.4
```

You can monitor the application by following the logs of the victim:

```
docker logs --follow assignment_5_dns_victim
```

Hint: You can time your DNS request correctly by making a DNS request to the resolver and inspecting the TTL field of the response, or you can wait for the TTL to expire and intercept the query to the authoritative DNS resolver.

Submission Instructions

Your script should run on the *attacker* container. You may not use external libraries or tools different from those installed on the provided *attacker* container.